

Object-Oriented Programming and Design

Project SI – API documentation

Nude Eye Project

Digital Engineering School
La Salle – Ramon Llull University

INDEX

1 - Introduction	1
2 - HTTP Protocol	2
2.1 - Requests.....	2
2.2 - Responses	3
2.3 - URL format.....	3
3 - Project API specification.....	5
3.1 - Products.....	5
3.1.1 - Get products.....	5
3.2 - Clients.....	6
3.2.1 - Create a new client.....	6
3.2.2 - Get clients.....	6
3.2.3 - Remove client	7
3.3 - Sales.....	7
3.4 - Providers.....	8
4 - HTTP requests using Java.....	9
4.1 - ApiHelper library.....	9
4.2 - HTTP connections with native Java	11
4.2.1 - Before Java 11 tool for HTTP protocol.....	11
4.2.2 - Java 11 tools for HTTP protocol.....	13

1 – INTRODUCTION

The Internet has become an omnipresent tool in our society. From the first message we send on Discord when waking up, to the submission we make on eStudy just before going to bed, our lives revolve around this communication system.

Specifically, most platforms with which we interact use the HTTP protocol to communicate the applications on our devices with the corresponding servers. As we don't want to be left behind, the OOPD subject has also decided to use this protocol to communicate with the API of the digital purchasing system we are implementing.

In this document, you will find an introduction to the HTTP protocol, as well as a detailed specification of the system's API, which has already been designed and developed by the subject team. Finally, various ways of working with this protocol in Java are presented.

2 – HTTP PROTOCOL

The HTTP protocol is one of the foundations of the modern web. When we access a web page with our preferred browser, we make HTTP requests, even if it is transparent to us. Similarly, the vast majority of requests made by mobile device applications also follow this protocol.

Specifically, it is an application-layer protocol (layer 7 of the OSI model) for client-server communication. This directionality is important because all communication will take place in the form of specific requests from the client to the server, which will respond synchronously.

While the protocol has evolved, incorporating more advanced techniques, the general idea of its operation remains as can be seen in Figure 1.

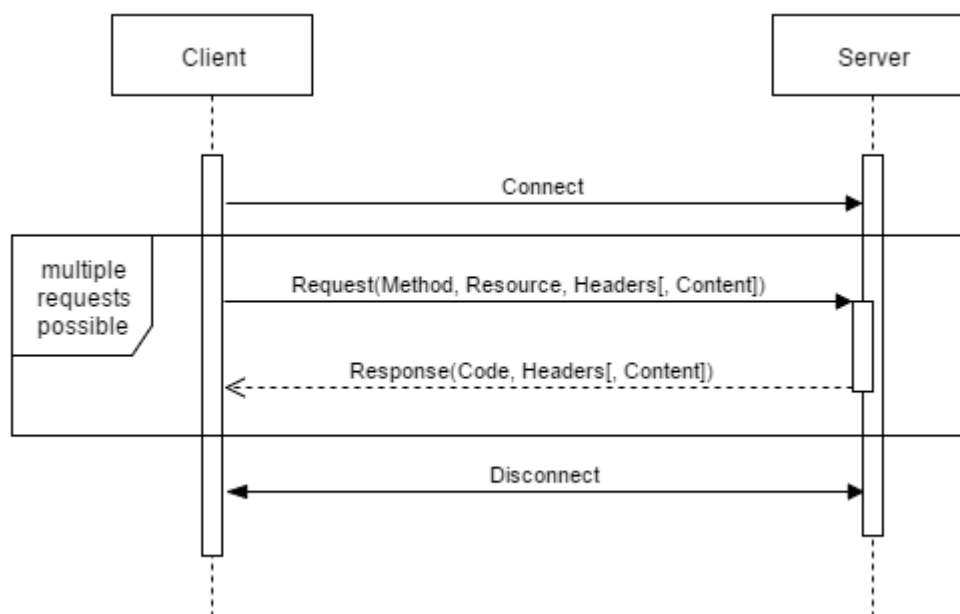


Figure 1: Diagram of a client-server protocol. Font: O'Reilly, Mastering Internet of Things

Although the protocol has evolved since its invention, to incorporate new modern techniques, the general idea of a Client-Server protocol remains the same. Figure 1 displays an example of its functioning.

2.1 – Requests

HTTP requests are frames sent by the client to the server. They are made up of four notable parts, among others:

- **Method:** The method, a word representing the goal of the request. The most common ones are GET (to obtain resources from the server) and POST (to create a new one), but there are others such as DELETE (to delete an existing one). For more information, check: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
- **Path:** The path of the resource to which the request applies. Although the frame simplifies it, it corresponds to a URL on the server. This point is detailed at the end of this chapter.

- **Headers:** Headers that the client can include (or not) to provide extra information to the server about its request, such as the format of the body. For more information, check the resource: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
- **Body:** The body of the request, which can be empty. It is usually text in a format known to both the client and the server. Nowadays, it is quite common to use the JSON format. The client usually sends information only in those methods that require it (for example, the information of the resource to be created in a POST).

2.2 – Responses

An HTTP response is the frame that the server sends to the client after receiving a request. They are made up of three notable parts, among others:

- **Status code:** A number indicating the status of the response. In general, it indicates whether the request has been successfully fulfilled. For more information, check the resource: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- **Headers:** Headers that the server can include (or not) to provide us with extra information about its configuration, the format of the response, or other details. For more information, check the resource: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
- **Body:** The body of the response, which can be empty. It is usually text in a format known to both the server and the client. Nowadays, it is quite common to use the JSON format.

2.3 – URL format

As explained, HTTP requests include the path on which they are being performed. Conceptually, this path corresponds to a server URL, following a standard format. In Figure 2, you can see an example.

```
https://api.server.com/endpoint/path?id=2&name=example&public=true
```

Figure 2: Example of a URL

Although the syntax of URLs allows for adding more elements, this example visualizes the most relevant parts of the HTTP protocol, especially for APIs. Here are the explanations for each part:

- **Protocol (https://):** Usually, it will be http or https, but it's good to keep in mind that other protocols are used in different contexts. Remember that in the web world, it's preferable to work with https to avoid data interception.
- **Host (api.server.com):** The domain name of the server we are communicating with (with its different parts, such as the subdomain).
- **Endpoint (/endpoint/path):** The location (path) within the server to which we are sending the request. It's a series of segments separated by the / character. The next chapter describes the available endpoints in the API that you need to use, as well as their functioning.

- **Query (?id=2&name=example&public=true):** Optionally, a URL can contain extra information for the server. Specifically, you can add as many parameters as you want after the ?. Each parameter is specified with its name, followed by = and its value (which can be of any type). Parameters are separated by the & character, and their order doesn't matter.

Note that in a URL, there can be no special characters, so you need to perform the process known as URL encoding. You can find more information in the resource: [Percent Encoding](#)

3 – PROJECT API SPECIFICATION

The project's final stage requires that your program (client) establishes a connection with the API designed by the subject (server). The base URL of this API can be found in Figure 3. If you access it from the browser, you can view its documentation in a format slightly different from this document.

```
https://balandrau.salle.url.edu/dpoo
```

Figure 3: Base URL of the project's API.

Note that you can only access the balandrau server if you are connected to the campus eduroam network or if you connect to the university's VPN. You can find more information on how to do this in the relevant document on eStudy.

To implement the requirements outlined in the corresponding statement, you have a set of endpoints, which are explained below.

3.1 – Products

To manage the products of the project, you will need to provide the word **shared** as part of the URL for the endpoint you are using. This is because these products will be shared by everyone and unmodifiable as well.

3.1.1 – Get products

The **/shared/products** endpoint responds to GET requests with the list of existing products. The response body follows the JSON format used in its creation.

This endpoint accepts parameters in the form of queries to filter products based on their attributes. If the attribute is a String, case-insensitive and partial equality logic (contains) will be applied. In case there is more than one parameter, the endpoint will return products that satisfy all conditions (as if it were an AND operation).

Additionally, there is a sub-endpoint to retrieve the product at a specific position in the list (starting from 0). If the position does not exist, it returns an error in JSON format.

Figure 4 illustrates the three mentioned scenarios with three different URLs. Feel free to try them in your browser to see the results, but keep in mind that to receive values, you must have created at least one product.

```
GET https://balandrau.salle.url.edu/dpoo/shared/products
GET https://balandrau.salle.url.edu/dpoo/shared/products?product_name=Lux
GET https://balandrau.salle.url.edu/dpoo/shared/products/2
```

Figure 4: URL examples for the get products endpoint.

3.2 – Clients

To manage your clients, you will need to provide your group identifier as part of the URL for the endpoint you are using. Throughout the document, this value will be referred to as **{id}**. The group identifier corresponds to the value you chose in the eStudy tool, for example, **Project-SI-455**.

3.2.1 – Create a new client

To create a new client, you need to make a **POST** request to the **/ {id} /clients** endpoint. The body of this request must contain the client data in the JSON format specified in the corresponding prompt. Figure 5 shows an example of this endpoint, including the body. The correct value to provide for the "Content-Type" header is also included, although this API does not require it.

```
POST https://balandrau.salle.url.edu/dpoo/P1-Example/clients
Headers: Content-Type="application/json"

Body: {
  "client_id": 20,
  "client_type": "corporate",
  "cif": "F99999999",
  "contact_name": "Generic Name",
  "billing_address": "C/ Llu  a 450, 08013 Barcelona",
  "mailing_address": "C/ St. Marc 89, 08024 Barcelona",
  "phone_numbers": [
    { "country_prefix": "+34", "number": "867 345 123" }
  ]
}
```

Figure 5: Example of the “create client” endpoint, including the optional headers and mandatory body.

The **/ {id} /clients** endpoint responds to GET requests with your list of clients. The response body follows the JSON format used in its creation.

This endpoint accepts parameters in the form of queries to filter clients based on their attributes. If the attribute is a String, case-insensitive and partial equality logic (contains) will be applied. In case there is more than one parameter, the endpoint will return clients that satisfy all conditions (as if it were an AND operation).

Additionally, there is a sub-endpoint to retrieve the client at a specific position in the list (starting from 0). If the position does not exist, it returns an error in JSON format.

Figure 6 illustrates the three mentioned scenarios with three different URLs. Feel free to try them in your browser to see the results, but keep in mind that to receive a response, you must have at least one client.

```
GET https://balandrau.salle.url.edu/dpoo/P1-Example/clients
GET https://balandrau.salle.url.edu/dpoo/P1-Example/clients?client_id=2
GET https://balandrau.salle.url.edu/dpoo/P1-Example/clients/5
```

Figure 6: URL examples for the get clients endpoint.

3.2.3 - Remove client

The deletion of clients will work similarly to the “get” query, but by accepting DELETE requests at the corresponding endpoint.

The URL with query parameters will delete clients that satisfy all the established conditions, but in this case, the check will be exact (case-sensitive and equals). If no client satisfies the conditions, an error in JSON format will be returned. Similarly, an error will be returned if no parameters are provided.

Clients can also be deleted based on their position in the list (starting from 0).

Figure 7 exemplifies the two mentioned scenarios with two different URLs.

```
DELETE https://balandrau.salle.url.edu/dpoo/P1-Example/clients?client_id=2
DELETE https://balandrau.salle.url.edu/dpoo/P1-Example/clients/3
```

Figure 7: URL examples of the remove clients endpoint.

3.3 - Sales

The endpoints for creating, querying, and deleting sales are equivalent to those of clients. Figure 8 lists all the corresponding URLs in the same order as detailed in the previous section.

```
POST https://balandrau.salle.url.edu/dpoo/P1-Example/sales
Headers: Content-Type="application/json"
Body: { ... }

GET https://balandrau.salle.url.edu/dpoo/P1-Example/sales
GET https://balandrau.salle.url.edu/dpoo/P1-Example/sales?client_id=2
GET https://balandrau.salle.url.edu/dpoo/P1-Example/sales/0

DELETE https://balandrau.salle.url.edu/dpoo/P1-Example/sales?client_id=2
DELETE https://balandrau.salle.url.edu/dpoo/P1-Example/sales/3
```

Figure 8: URL examples of endpoints available to manage the sales.

Note that the API is programmed to check the body’s JSON format but does not perform any checks on the business logic.

3.4 – Providers

The endpoints for querying providers are equivalent to those of products. Figure 9 lists all the corresponding URLs in the same order as detailed in the previous section.

```
GET https://balandrau.salle.url.edu/dpoo/shared/providers
GET https://balandrau.salle.url.edu/dpoo/shared/providers?phone=345
GET https://balandrau.salle.url.edu/dpoo/shared/providers/0
```

Figure 9: URL examples of endpoints available to manage the providers.

4 - HTTP REQUESTS USING JAVA

In this section, two ways of making HTTP requests in Java are explained. The first one involves using an auxiliary library provided by the subject, while the second one uses the default tools included in Java. Feel free to use the approach you find more suitable.

4.1 - ApiHelper library

To ease interaction with the system's API, you can use the auxiliary library ApiHelper, which has methods to abstract GET, POST, and DELETE requests. The JavaDoc is also included, detailing the functionality of each method. In Figure 8, you can see the UML class diagram for the library.

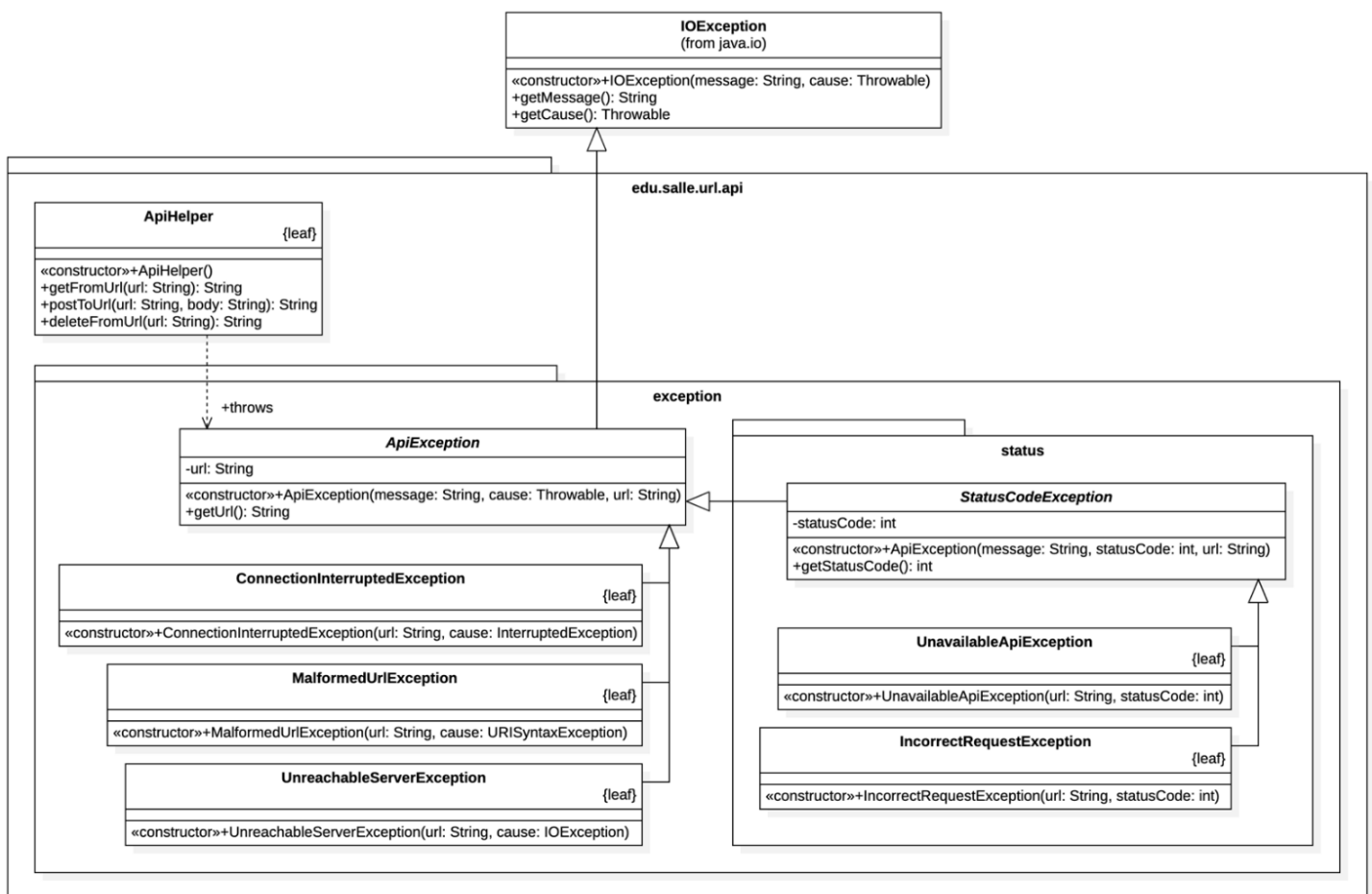


Figure 1: UML class diagram of the ApiHelper library.

ApiHelper is the main class for this library. However, it's important to note that its methods can throw exceptions specific to different reasons. While it's recommended to handle errors with the general **ApiException** class, you can use its subclasses to distinguish between specific causes:

- **ConnectionInterruptedException**: Thrown when the connection to the API is interrupted.
- **MalformedURLException**: Thrown when a syntax error is detected in the URL.

- **UnreachableServerException:** Thrown when the computer cannot connect to the API server. The server may be down, or you might not be connected through eduroam or the VPN.
- **StatusCodeException:** Abstract class that groups all exceptions where a response from the server has been received, but the **status code** that you get indicates that the request has not been successful. It has the following subclasses:
 - **IncorrectRequestException:** Thrown when the request you are sending the API is not correct. For instance, it can happen when trying to access an endpoint that does not exist. We recommend you to investigate the meaning behind the status code that you get (included in the Exception message) in order to find the error cause.
 - **UnavailableApiException:** Thrown when the server is available but the API is not. This situation should not happen, so if you find this issue please contact the subject team.

To work with the ApiHelper you can use the methods described in the class diagram:

- **ApiHelper:** The class constructor. It checks if a connection to the API can be established. Throws an ApiException if it encounters a problem.
- **getFromUrl:** Method for making a GET request to the API. It takes the URL to query and returns its contents. It can throw an ApiException in the cases mentioned earlier.
- **postToUrl:** Method for making a POST request to the API. It takes the URL to make the request and the content to send. It returns the server's response if there is one. It can throw an ApiException in the cases mentioned earlier.
- **deleteFromUrl:** Method for making a DELETE request to the API. It takes the URL to make the request to and returns the server's response if there is one. It can throw an ApiException in the cases mentioned earlier.

4.2 – HTTP connections with native Java

This section describes native tools for making HTTP requests in Java as an alternative to the ApiHelper library.

Initially, to implement a Java client that made HTTP requests, one had to use general-purpose classes such as *URLConnection* or *BufferedReader*. While this approach is still valid, it is quite manual and often does not accurately represent what we want to do.

To address these issues, Java 11 introduced a set of more specific-purpose classes like *HttpClient* or *HttpRequest*. The goal of these new classes is to encapsulate the necessary code to create and manage HTTP connections, abstracting us from its "low-level" details.

The following sections present examples of how to make GET and POST requests with each of these sets of tools. You can explore how a DELETE request would be performed.

4.2.1 – Before Java 11 tool for HTTP protocol

Figure 9 presents an example of a GET request using older (pre-Java 11) tools. In Figure 10, you can see an example of a POST request.

```
private String getStringFromUrl(String url) {
    try {
        // Define the URL, its corresponding connection and set up
        // a BufferedReader to read from it
        URL auxUrl = new URL(url);
        URLConnection con = auxUrl.openConnection();
        InputStreamReader aux = new InputStreamReader(con.getInputStream());
        BufferedReader in = new BufferedReader(aux);

        // Consume the reader
        StringBuilder stringBuilder = new StringBuilder();
        String inputLine;
        while ((inputLine = in.readLine()) != null) {
            stringBuilder.append(inputLine);
        }

        // Close the reader
        in.close();
        return stringBuilder.toString();
    } catch (IOException e) {
        // Note: This IOException encompasses a possible MalformedURLException
        // TODO manage properly.
        System.err.println(e.getMessage());
        return "";
    }
}
```

Figure 2. Example of reading the contents of a URL using general-purpose tools in Java, equivalent to a GET request.

```
private void postStringToUrl(String url, String body) {
    try {
        // Define the URL and its corresponding HTTP connection, set up
        // the method and establish that we want to output data as well
        // as to have specific headers (properties)
        URL auxUrl = new URL(url);
        HttpURLConnection con = (HttpURLConnection) auxUrl.openConnection();
        con.setDoOutput(true);
        con.setRequestMethod("POST");
        con.setRequestProperty("Content-Type", "application/json");

        // Set up an OutputStreamWriter to send the body to the connection
        OutputStreamWriter out = new OutputStreamWriter(con.getOutputStream());
        out.write(body);
        out.close();

        // Set up and consume a BufferedReader to read from the connection, in
        // case the API returns something (we could also choose to ignore it)
        InputStreamReader aux = new InputStreamReader(con.getInputStream());
        BufferedReader in = new BufferedReader(aux);

        StringBuilder stringBuilder = new StringBuilder();
        String inputLine;
        while ((inputLine = in.readLine()) != null) {
            stringBuilder.append(inputLine);
        }

        // Close the reader
        in.close();
        System.out.println(stringBuilder);
    } catch (IOException e) {
        // Note: This IOException encompasses a possible MalformedURLException
        // TODO manage properly.
        System.err.println(e.getMessage());
    }
}
```

Figure 3. Example of writing content to a URL using general-purpose tools in Java, equivalent to a POST request.

4.2.2 - Java 11 tools for HTTP protocol

Figure 4 exemplifies a GET request using modern tools introduced in Java 11. Likewise, Figure 5 exemplifies a POST request.

```
private String getStringFromUrl(String url) {
    try {
        // Define the request
        // The default method is GET, so we don't need to specify it
        // (but we could do so by calling .GET() before .build())
        // The HttpRequest.Builder pattern offers a ton of customization for
        // the request (headers, body, HTTP version...)
        HttpRequest request = HttpRequest.newBuilder()
            .uri(new URI(url))
            .build();

        // We have to create a client to send the request
        // This could also be done by using the HttpClient.Builder pattern
        // if we needed to customize it, but we don't
        HttpClient client = HttpClient.newHttpClient();

        // We use the default BodyHandler for strings
        // (so we can get the body of the response as a String)
        // Note we could also send the request asynchronously, but things
        // would escalate in terms of coding complexity
        HttpResponse<String> response = client.send(
            request,
            HttpResponse.BodyHandlers.ofString()
        );

        // Just return the body
        return response.body();
    } catch (URISyntaxException | IOException | InterruptedException e) {
        // TODO manage properly.
        System.err.println(e.getMessage());
        return "";
    }
}
```

Figure 4. Example of a POST request using modern tools in Java.

```
private void postStringToUrl(String url, String body) {
    try {
        // Define the request
        // In this case, we have to use the .POST() and .headers() methods
        // to define what we want (to send a string containing JSON data)
        HttpRequest request = HttpRequest.newBuilder()
            .uri(new URI(url))
            .headers("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();

        // We have to create a client to send the request
        // This could also be done by using the HttpClient.Builder pattern
        // if we needed to customize it, but we don't
        HttpClient client = HttpClient.newHttpClient();

        // We could use a BodyHandler that discards the response header, but
        // here we print the API's response (and could process it if needed)
        // Note we could also send the request asynchronously, but things
        // would escalate in terms of coding complexity
        HttpResponse<String> response = client.send(
            request,
            HttpResponse.BodyHandlers.ofString()
        );
        System.out.println(response.body());

        // Alternative we could use to discard the response:
        // client.send(request, HttpResponse.BodyHandlers.discarding());

    } catch (URISyntaxException | IOException | InterruptedException e) {
        // TODO manage properly.
        System.err.println(e.getMessage());
    }
}
```

Figure 5. Example of a POST request using modern tools in Java.